

I'm building amaarshop, a lightweight Shopify-style platform for small Bangladeshi businesses who currently sell on Facebook. Each seller gets their own storefront at `/s/{slug}` which they paste into their Facebook page's "Shop Now" button. MVP scale: <100 shops, Bangladesh only, cash-on-delivery only (no online payments).

Stack: Go backend (net/http router, sqlx, JWT, PostgreSQL 16), React + Vite + TypeScript frontend (Tailwind, TanStack Query, react-i18next, Recharts), everything dockerized.

Backend follows strict clean architecture: handler → service → repository → database. Dependencies point inward only. Domain layer has zero external imports. Repository interfaces live in `internal/repository/`, Postgres implementations in `internal/repository/postgres/`. Services take repository interfaces via constructor injection. All wiring happens in `cmd/api/main.go`. Domain errors are sentinel errors mapped to HTTP responses by a single function in the handler layer. Transactions managed at service layer via a DBTX interface that both `*sql.DB` and `*sql.Tx` satisfy.

Package layout:

```
backend/
├── cmd/api/main.go
├── internal/
│   ├── config/ domain/ handler/{http,dto}/ service/
│   ├── repository/{postgres}/ storage/{local}/ auth/ platform/{database,logger}/
├── migrations/ scripts/ Dockerfile Dockerfile.dev Makefile
```

Frontend has three route groups sharing one component library: marketing site, seller dashboard (`/dashboard/*`), customer storefront (`/s/{slug}/*`). All strings go through react-i18next (en and bn). Mobile-first, 360px baseline.

Security: bcrypt, JWT access + httpOnly refresh cookies, CSRF on mutations, rate limiting on auth, parameterized queries, image upload validation.

Canonical references: Every sub-prompt must refer to `docs/SCHEMA.md` (from Sub-Prompt 2) and `docs/API.md` (from Sub-Prompt 3) as the source of truth. If

implementing a feature reveals a need to change the schema or API, update those docs first with a migration, then implement.

Sub-Prompt 1 — Project Skeleton & Docker Infrastructure

Set up the repository skeleton and Docker infrastructure. No business features — just scaffolding.

Deliverables:

1. Repo layout with backend/, frontend/, docs/, and root-level `docker-compose.yml`, `docker-compose.prod.yml`, `.env.example`, `.dockerignore`, `Makefile`, `README.md`, `PROGRESS.md`.
2. backend/ with the package layout from the shared context. Placeholder `doc.go` files per package. `cmd/api/main.go` starts a chi server on a configurable port, registers GET /health returning {"status": "ok"}, sets up slog JSON logging, and shuts down gracefully on SIGINT/SIGTERM.
3. backend/Dockerfile (multi-stage: golang:1.23 builder → distroless or alpine final) and backend/Dockerfile.dev using air for hot reload. The dev Dockerfile copies go.mod/go.sum and runs go mod download before copying source.
4. frontend/ initialized with Vite + React + TypeScript + Tailwind + react-router + TanStack Query + react-i18next. Single placeholder route showing "ShopLink" with a working en/bn language toggle.
5. frontend/Dockerfile.dev running Vite dev server with mounted source.
6. docker-compose.yml with: postgres (PostgreSQL 16, named volume, healthcheck), backend (depends on postgres healthy, uses Dockerfile.dev, mounts source, named volumes for Go build/module cache), frontend (uses Dockerfile.dev, mounts source), migrate (one-shot, golang-migrate), adminer.
7. docker-compose.prod.yml override: built images, no source mounts, restart policies, nginx reverse-proxying backend and frontend.
8. Makefile with: up, down, logs, migrate-up, migrate-down, migrate-create name=..., seed, test, build-prod.

9. `.env.example` documenting every variable (DB creds, JWT secret, admin seed, upload dir, port, etc.).
10. `README.md` with one-command setup. `PROGRESS.md` with a checklist of all 13 sub-prompts.

Acceptance: `make up` brings up the entire stack, `curl`

`localhost:8080/health` returns OK, the frontend renders at its dev port, the language toggle works.

Sub-Prompt 2 — Database Design & Migrations (NEW)

Design the complete database schema for ShopLink in one pass. Create migrations for every table that will exist across the full MVP. No Go code beyond the migration files — this sub-prompt is about locking down the data model before any feature work.

Deliverables:

1. **docs/SCHEMA.md** — the canonical schema document. For each table: purpose, columns with types and constraints, indexes, foreign keys, sample rows. Include an ER diagram (Mermaid is fine). This file is referenced by every subsequent sub-prompt as the source of truth.
2. **Migration files** in `backend/migrations/` using `golang-migrate` naming convention (`000001_create_users.up.sql` / `.down.sql`, etc.). One migration per logical table group. Every up must have a working down.
3. **Tables to create:**
 - **users** — `id` (uuid pk), `email` (citext unique not null), `password_hash` (text not null), `is_admin` (bool default false), `created_at`, `updated_at`.
 - **shops** — `id` (uuid pk), `owner_user_id` (uuid unique fk → users, cascade), `slug` (citext unique not null, check: lowercase alphanumeric + hyphens, length 3–40), `name`, `description`, `logo_url`, `banner_url`, `contact_phone`, `is_suspended` (bool

default false), created_at, updated_at. Unique on owner_user_id enforces one-shop-per-user for MVP.

- **shop_delivery_settings** — shop_id (uuid pk, fk → shops, cascade), cod_enabled (bool default true), delivery_charge (numeric(10,2) default 0 check ≥ 0), free_delivery_threshold (numeric(10,2) nullable check > delivery_charge when not null), advance_payment_required (bool default false), advance_payment_instructions (text), delivery_areas (text[] default '{}'), updated_at. One row per shop; created on shop creation via trigger or service.
- **categories** — id (uuid pk), shop_id (uuid fk → shops, cascade), name (text not null), created_at. Unique on (shop_id, lower(name)). Index on shop_id.
- **products** — id (uuid pk), shop_id (uuid fk → shops, cascade), category_id (uuid nullable fk → categories, set null), name (text not null), description (text), price_bdt (numeric(10,2) not null check > 0), stock (int not null default 0 check ≥ 0), is_active (bool default true), is_archived (bool default false), created_at, updated_at. Indexes: (shop_id, is_active, is_archived), (shop_id, category_id), (shop_id) where is_archived = false, plus a text search index on name using pg_trgm for search.
- **product_images** — id (uuid pk), product_id (uuid fk → products, cascade), url (text not null), sort_order (int default 0), created_at. Check constraint or app-level enforcement: max 5 per product. Index on (product_id, sort_order).
- **product_variants** — id (uuid pk), product_id (uuid fk → products, cascade), name (text not null), sku (text), price_override (numeric(10,2) nullable), stock (int not null default 0 check ≥ 0), created_at. **Table exists from day one even though UI doesn't expose it in MVP** — prevents schema churn later.
- **orders** — id (uuid pk), shop_id (uuid fk → shops, restrict), customer_name (text not null), customer_phone (text not null), delivery_address (text not null), delivery_area (text not null), note (text), subtotal_bdt (numeric(10,2) not null), delivery_charge_bdt (numeric(10,2) not null default 0), total_bdt (numeric(10,2) not null), status (text not null check in 'pending','confirmed','shipped','delivered','cancelled', default

'pending'), advance_payment_required (bool default false), advance_payment_received (bool default false), cancelled_reason (text), created_at, updated_at. Indexes: (shop_id, status, created_at desc), (shop_id, customer_phone), (created_at desc).

- **order_items** — id (uuid pk), order_id (uuid fk → orders, cascade), product_id (uuid fk → products, restrict), product_name_snapshot (text not null), unit_price_snapshot_bdt (numeric(10,2) not null), quantity (int not null check > 0), line_total_bdt (numeric(10,2) not null), created_at. Product name and price are **snapshotted** at order time so future edits don't mutate history. Index on order_id.

4. Shared conventions:

- All ids are uuid with gen_random_uuid() default (enable pgcrypto extension).
- All tables have created_at timestampz not null default now(). Tables that can be updated also have updated_at timestampz not null default now(). Use a single set_updated_at() trigger function reused across all tables.
- Money fields are numeric(10,2) — never float.
- Enable extensions: pgcrypto, citext, pg_trgm.
- Every fk has an explicit on delete action documented in SCHEMA.md.

5. **Tenant isolation rule** (document at the top of SCHEMA.md): every query for tenant-scoped data (categories, products, orders, etc.) **must** include shop_id in the WHERE clause. Repository implementations enforce this; reviewers should reject any query that doesn't. Optionally, add a check function / comment block explaining why row-level security was not used (keeping MVP simple).

6. **Seed migration (separate, not run automatically):** scripts/seed.sql or scripts/seed.go creating one demo seller, one shop (demo-shop), three categories, six products with images, and three sample orders in different statuses. Invoked via make seed. **Never runs in production** — guard with an env flag.

Acceptance: make migrate-up applies all migrations cleanly against a fresh database. make migrate-down reverses them cleanly. make seed populates the

demo data. docs/SCHEMA.md is complete and includes the ER diagram. Adminer shows all tables with correct constraints.

Sub-Prompt 3 — API Contract (NEW)

Write the complete API contract for ShopLink as docs/API.md before implementing any handlers. This becomes the reference every later sub-prompt consults.

Deliverables:

1. **docs/API.md** organized by resource. For every endpoint document: method, path, auth requirement (public / seller / admin), request body schema, response body schema, status codes, error codes, example request/response. Use a consistent JSON envelope: { "data": ... } for success, { "error": { "code": "...", "message": "..." } } for errors.
2. **Endpoints to specify (grouped by sub-prompt they'll be implemented in):**

Auth (Sub-Prompt 4):

- POST /api/auth/signup, POST /api/auth/login, POST /api/auth/refresh, POST /api/auth/logout, GET /api/auth/me.

3. **Shops (Sub-Prompt 5):**

- POST /api/shops — create own shop (one per user).
- GET /api/shops/me, PATCH /api/shops/me.
- POST /api/shops/me/logo, POST /api/shops/me/banner.
- GET /api/shops/check-slug?slug=...
- GET /api/shops/me/delivery-settings, PUT /api/shops/me/delivery-settings.
- Public: GET /api/shops/by-slug/{slug}, GET /api/shops/by-slug/{slug}/delivery-settings.

4. **Products & Categories (Sub-Prompt 6):**

- Seller: GET/POST /api/shops/me/categories, PATCH/DELETE /api/shops/me/categories/{id}.
- Seller: GET/POST /api/shops/me/products, GET/PATCH/DELETE /api/shops/me/products/{id}, POST /api/shops/me/products/{id}/archive, POST /api/shops/me/products/{id}/images, DELETE /api/shops/me/products/{id}/images/{imageID}, PATCH /api/shops/me/products/{id}/images/reorder.
- Public: GET /api/shops/by-slug/{slug}/products (with q, category_id, page, page_size), GET /api/shops/by-slug/{slug}/products/{id}, GET /api/shops/by-slug/{slug}/categories.

5. Orders (Sub-Prompt 7):

- Public: POST /api/shops/by-slug/{slug}/orders, GET /api/orders/{id}/lookup?phone=...
- Seller: GET /api/shops/me/orders (filters: status, from, to, q for phone), GET /api/shops/me/orders/{id}, PATCH /api/shops/me/orders/{id}/status, POST /api/shops/me/orders/{id}/mark-advance-received.

6. Analytics (Sub-Prompt 8):

- GET /api/shops/me/stats/today, GET /api/shops/me/stats/range, GET /api/shops/me/stats/top-products.

7. Facebook/QR (Sub-Prompt 9):

- GET /api/shops/me/qr.

8. Admin (Sub-Prompt 10):

- GET /api/admin/shops, GET /api/admin/shops/{id}, POST /api/admin/shops/{id}/suspend, POST /api/admin/shops/{id}/unsuspend, GET /api/admin/stats.

9. Health:

- GET /health, GET /ready.

10. Cross-cutting conventions to document:

- Authentication: Authorization: Bearer <access_token> header; refresh token in httpOnly cookie.
- Error codes: canonical list (e.g., validation_error, unauthorized, forbidden, not_found, slug_taken, insufficient_stock, invalid_status_transition, checkout_disabled, rate_limited).
- Pagination: page, page_size query params; response includes { data, pagination: { page, page_size, total, total_pages } }.
- Suspended-shop rule: **every public by-slug endpoint returns 404 if the shop is suspended**. Document this once, enforced in a middleware or shop-loader helper.
- Customer phone lookup: FindByIDAndPhone uses constant-time comparison and normalizes phone before comparing (strip spaces, leading +88, etc.).
- Image upload limits: max 2MB, JPEG/PNG/WebP only.

Acceptance: docs/API.md lists every endpoint the MVP will ever expose, with consistent shape and clear error codes. Later sub-prompts consult this file instead of reinventing endpoints.

Sub-Prompt 4 — Auth Foundation (was Sub-Prompt 2)

Implement authentication per docs/API.md auth section and docs/SCHEMA.md users table.

Backend:

1. internal/domain/user.go — User struct matching the schema, domain errors ErrUserNotFound, ErrEmailAlreadyExists, ErrInvalidCredentials.
2. internal/repository/user_repository.go interface + internal/repository/postgres/user_repo.go Postgres implementation. Methods: Create, FindByEmail, FindByID.
3. internal/auth/ — bcrypt helpers, JWT creation and validation (access: 15 min, refresh: 7 days), secret from env.

4. `internal/service/auth_service.go` — Signup, Login, Refresh, Me. Constructor takes `UserRepository` interface.
5. `internal/handler/http/auth_handler.go` implementing the endpoints from `docs/API.md`. Refresh token as `httpOnly Secure SameSite=Strict` cookie; access token in JSON.
6. `internal/handler/http/middleware/auth.go` — validates access token, injects user ID into context.
7. `internal/handler/http/middleware/ratelimit.go` — in-memory token bucket per IP, applied to `/api/auth/login` and `/api/auth/signup`.
8. Centralized domain-error-to-HTTP response mapper in `internal/handler/http/errors.go`. All future handlers use it.
9. Seed the admin user from `ADMIN_EMAIL` / `ADMIN_PASSWORD` env vars on startup if no admin exists.
10. Unit tests for `auth_service` with a mocked user repo.

Frontend:

1. `/signup`, `/login` pages with validation, errors displayed in active language.
2. Auth context / TanStack Query hook that stores access token in memory, auto-refreshes on 401 via cookie.
3. Protected route wrapper for `/dashboard/*`. Logout button.

Acceptance: Signup → login → hit a protected placeholder → transparent refresh → logout all work. Rate limiting blocks brute force. Admin user is seeded on first run.

Sub-Prompt 5 — Shop Creation, Settings, and Delivery Configuration (merges old 3 + 4)

Implement the shop entity and per-shop delivery/COD settings as one vertical slice. Per `docs/SCHEMA.md` the shops and `shop_delivery_settings` tables are already created.

Backend:

1. `internal/domain/shop.go` — Shop struct, slug validation, domain errors `ErrShopNotFound`, `ErrSlugTaken`, `ErrShopAlreadyExists`, `ErrNotShopOwner`.
2. `internal/domain/delivery_settings.go` — `DeliverySettings` struct, validation (charge ≥ 0 , threshold > charge when set, areas non-empty if COD enabled).
3. `internal/repository/shop_repository.go` interface + Postgres impl: `Create`, `FindByID`, `FindBySlug`, `FindByOwnerID`, `Update`, `IsSlugAvailable`.
4. `internal/repository/delivery_settings_repository.go` interface + Postgres impl: `Get(shopID)`, `Upsert(settings)`.
5. `internal/storage/storage.go` interface + `internal/storage/local/` implementation writing to configurable upload directory served at `/uploads/*`. Validation: JPEG/PNG/WebP, max 2MB.
6. `internal/service/shop_service.go` — methods matching the API contract. Business rules: slug available, only owner can mutate, delivery settings row created automatically on shop creation.
7. **Shop loader helper** in `internal/handler/http/shop_loader.go` — resolves a shop by slug and returns 404 if it's suspended. Used by every public `by-slug` endpoint across all sub-prompts.
8. Handlers per `docs/API.md`.

Frontend:

1. Post-login: if user has no shop, redirect to `/dashboard/setup`.
2. 3-step wizard:
 - Step 1: shop name + slug (live availability, autosuggest from name).
 - Step 2: logo, banner, description, contact phone.
 - Step 3: delivery settings — COD toggle, delivery charge, free-delivery threshold, advance-payment toggle + bKash/Nagad instructions textarea, delivery areas as tag input. Inline Bengali/English help text.
3. `/dashboard/settings` — edit branding.
`/dashboard/settings/delivery` — edit delivery settings.

Acceptance: Seller can complete the wizard end-to-end. Public `by-slug` endpoints respect the suspension flag (returning 404 for suspended shops). Delivery settings round-trip correctly.

Sub-Prompt 6 — Product Management (was 5)

Implement categories, products, variants, and images per docs/SCHEMA.md and docs/API.md.

Backend:

1. internal/domain/product.go, internal/domain/category.go, internal/domain/product_variant.go (struct exists even if UI doesn't surface variants yet), internal/domain/product_image.go. Domain errors ErrProductNotFound, ErrCategoryNotFound, ErrInsufficientStock, ErrTooManyImages, ErrCategoryNotInShop.
2. CategoryRepository and ProductRepository interfaces + Postgres impls. Product repo: Create, Update, Archive, Delete, FindByID, ListByShop(shopID, filters, pagination), ListPublicByShopSlug(slug, filters, pagination). Image ops and variant ops live on ProductRepository.
3. **Every query must include shop_id** per the tenant isolation rule in docs/SCHEMA.md.
4. product_service and category_service with business rules: only owner can mutate, max 5 images, category must belong to same shop, price > 0, stock ≥ 0.
5. Handlers per docs/API.md. Public endpoints use the shop loader and return 404 for suspended shops.
6. Unit tests: stock validation, ownership checks, image limit, cross-shop category rejection.

Frontend (seller dashboard only):

1. /dashboard/products — paginated list with search (uses pg_trgm on backend), category filter, active/archived toggle.
2. /dashboard/products/new and /dashboard/products/{id}/edit — form with image upload (drag-drop, up to 5, reorderable), inline category creation.
3. /dashboard/categories — simple CRUD.

Acceptance: Seller can fully manage categories and products. Public endpoints return only active, non-archived products from non-suspended shops. Cross-shop access attempts return 404.

Sub-Prompt 7 — Public Storefront (was 6)

Build the customer-facing storefront. No customer accounts. Cart is client-side until checkout submission.

Backend: Only add whatever public endpoints from docs/API.md aren't yet implemented (e.g., GET /api/shops/by-slug/{slug}/categories). All endpoints must route through the shop loader.

Frontend (storefront route group /s/{slug}/*):

1. /s/{slug} — shop landing: banner, logo, description, search, category chips, product grid. Empty state if no products.
2. /s/{slug}/p/{productID} — product detail: gallery, name, price, description, quantity, "Add to Cart". Out-of-stock disables the button.
3. Cart drawer via floating icon. State in localStorage scoped per shop slug.
4. /s/{slug}/checkout — cart summary, delivery charge from settings + threshold, total. Form: name, phone (BD format), address, delivery area dropdown, optional note. If advance_payment_required, show bKash/Nagad instructions prominently with a "I understand" checkbox. If COD is disabled, replace the page with "Not currently taking orders" + contact phone.
5. All pages mobile-first — storefront is the most performance-critical surface.

Acceptance: Customer arriving from a Facebook Shop Now link can browse, add to cart, and reach checkout with settings applied correctly.

Sub-Prompt 8 — Order Creation & Management (was 7)

Implement orders per schema and API contract. Largest sub-prompt.

Backend:

1. `internal/domain/order.go`, `internal/domain/order_item.go`.
Status enum + transition map. Domain errors `ErrOrderNotFound`, `ErrInvalidStatusTransition`, `ErrCheckoutDisabled`, `ErrEmptyCart`, `ErrProductUnavailable`.
2. `OrderRepository` interface + Postgres impl. `Create(order, items)` in one transaction. `FindByID`, `FindByIDAndPhone` (**with phone normalization and constant-time compare**), `ListByShop`, `UpdateStatus`, `MarkAdvanceReceived`, `Stats`.
3. `internal/service/order_service.go`:
 - `CreateOrder(shopSlug, items, customerInfo)` — validates shop not suspended and COD enabled; loads products and verifies active, in same shop, sufficient stock; **recalculates prices server-side (never trusts client totals)**; snapshots product name and price into items; computes delivery charge; decrements stock; persists order + items in one transaction.
 - `UpdateStatus` — enforces allowed transitions: pending → confirmed → shipped → delivered, any non-delivered → cancelled. Cancelling a non-delivered order restores stock in the same transaction.
 - `MarkAdvanceReceived`, `LookupForCustomer`.
4. Handlers per `docs/API.md`.
5. Unit tests: total calculation, transition enforcement, stock decrement/restore, rejecting cross-shop items.

Frontend:

1. **Storefront:** wire checkout to POST the order. On success navigate to `/s/{slug}/order-confirmed/{orderId}` with order ID prominently shown, items, totals, delivery info, advance-payment instructions repeated if applicable, "screenshot this" hint. Add `/s/{slug}/order-lookup`.
2. **Seller dashboard:** `/dashboard/orders` table with filters + phone search. `/dashboard/orders/{id}` detail with status badge, action buttons for valid next states only, "Mark advance received" when applicable.

Acceptance: End-to-end: customer checks out → order in seller dashboard → seller progresses status → cancellation restores stock → customer looks up by ID + phone. Server-recalculated totals prevent price tampering.

Sub-Prompt 9 — Dashboard Overview & Analytics (was 8)

Dashboard home + analytics page.

Backend:

1. analytics_service (or methods on order_service): TodayStats, RangeStats (daily series), TopProducts.
2. Endpoints per docs/API.md.
3. In-memory cache with 5-minute TTL for aggregates.

Frontend:

1. /dashboard overview: cards for today's orders, pending, revenue today/week/month; pending orders quick list; top 5 products this month.
2. /dashboard/analytics: date range picker, line chart of orders/revenue (Recharts), top products bar chart.

Acceptance: Numbers match the orders table. Charts render on mobile.

Sub-Prompt 10 — Facebook Connect Guide & QR Code (was 9)

Manual "Facebook integration" + QR generation.

Backend: GET /api/shops/me/qr — generates a PNG QR of the shop URL, cached per shop.

Frontend:

1. /dashboard/connect: big "Copy shop URL" button, step-by-step Bengali + English guide with screenshots for adding the URL to a Facebook page's Shop Now CTA, downloadable QR, "Test it" link, optional **"Message to Order"** section explaining the Messenger fallback.
2. Dashboard overview shows a "Connect to Facebook" CTA until the seller visits /dashboard/connect at least once.

Acceptance: Seller can copy the URL, download QR, and follow the guide.

Sub-Prompt 11 — Admin Panel (was 10)

Minimal admin panel. `is_admin` column already exists per schema.

Backend:

1. Admin middleware requiring `is_admin`.
2. Endpoints per `docs/API.md` admin section.
3. Suspending a shop must cause all public `by-slug` endpoints to 404 immediately (already enforced by the shop loader).

Frontend:

1. `/admin` route group with admin-only wrapper.
2. `/admin/shops` — table with suspend/unsuspend actions. `/admin/stats` — platform totals.

Acceptance: Seeded admin logs in, suspends a shop, and the storefront becomes unreachable instantly.

Sub-Prompt 12 — Localization Pass & Mobile Polish (was 11)

Fill in every translation, polish mobile, add finishing touches.

Tasks:

1. Audit every page for hardcoded strings; move all to `en.json` and `bn.json`.
2. Translate to Bengali including form validation and API error messages (backend returns error codes; frontend maps to localized strings).
3. BDT currency helper, `Intl.DateTimeFormat` with locale.
4. Mobile audit at 360px — especially storefront, checkout, order detail.
5. Loading skeletons, empty states, shared toast component.
6. Set `<html lang>` based on active locale.

Acceptance: Monolingual Bengali user completes every flow without seeing English. Every screen works at 360px.

Sub-Prompt 13 — Production Deployment & Hardening (was 12)

Ship-ready.

Tasks:

1. Verify `docker-compose.prod.yml` builds production images and runs nginx in front.
2. Nginx: serve frontend static assets, proxy `/api` and `/uploads`, gzip, cache headers, max upload size matching backend.
3. HTTPS via Let's Encrypt (nginx-proxy + acme-companion, or Caddy as simpler alternative — document both).
4. `/health` and `/ready` (`/ready` checks DB).
5. Daily `pg_dump` sidecar writing to a mounted volume.
6. JSON logs to stdout; document shipping later.
7. Production checklist in README: env vars, migrations, admin seed, backup restore, deploy command.
8. Security pass: rate limiting, CSRF, upload validation, no secrets in git history.

Acceptance: Fresh VPS runs the entire stack from `git clone + make build-prod + docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`.
